Adversarial Game Playing

Min-Max with Alpha Beta Pruning

Admissibility of A*

- A* will never overestimate the cost of reaching the goal.
- The cost to reach goal is guaranteed to be lower or equal to the actual cost.
- This property ensures that the algorithm will eventually find the optimal solution, if one exists

Admissibility of A*

- A heuristic h(n) is **admissible** if for every node n, h(n) ≤ h*(n), where h*(n) is the **true/actual** cost to reach the goal state from n and h(n) is the estimated cost to reach the goal
- An admissible heuristic never overestimates the cost to reach the goal, i.e., it is **optimistic**
  - Example: hSLD(n) (never overestimates the actual road distance)

Theorem: If h(n) is admissible, A* using TREE-SEARCH is optimal

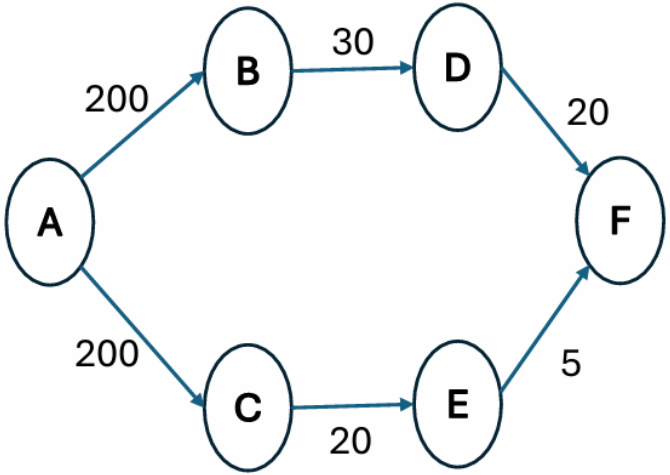
Admissibility of A*

- Find the shortest path
- Source = A
- Goal = F

- According UCS path cost

  A→ C → E → F = 225 **Best**

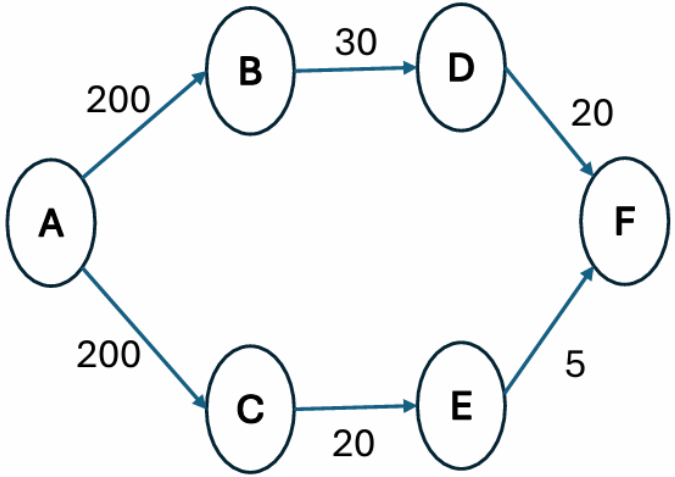
  A→ B → D → F = 250



Admissibility of A*

- Underestimated heuristic

City (n)	h(n)	City (n)	h(n)
A	35	D	10
B	20	E	5
C	25	F	0



- Apply A*

A → C → E → F

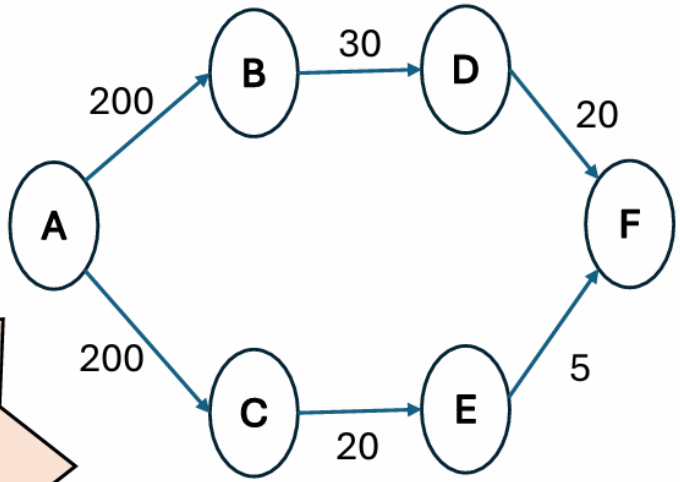
Admissibility of A*

- Overestimated heuristic

City (n)	h(n)	City (n)	h(n)
A	35	D	10
B	70	E	10
C	80	F	0

- Apply A*

A→ B → D → F

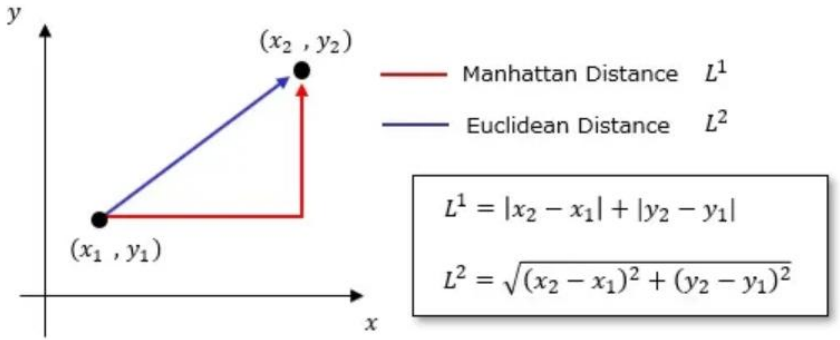


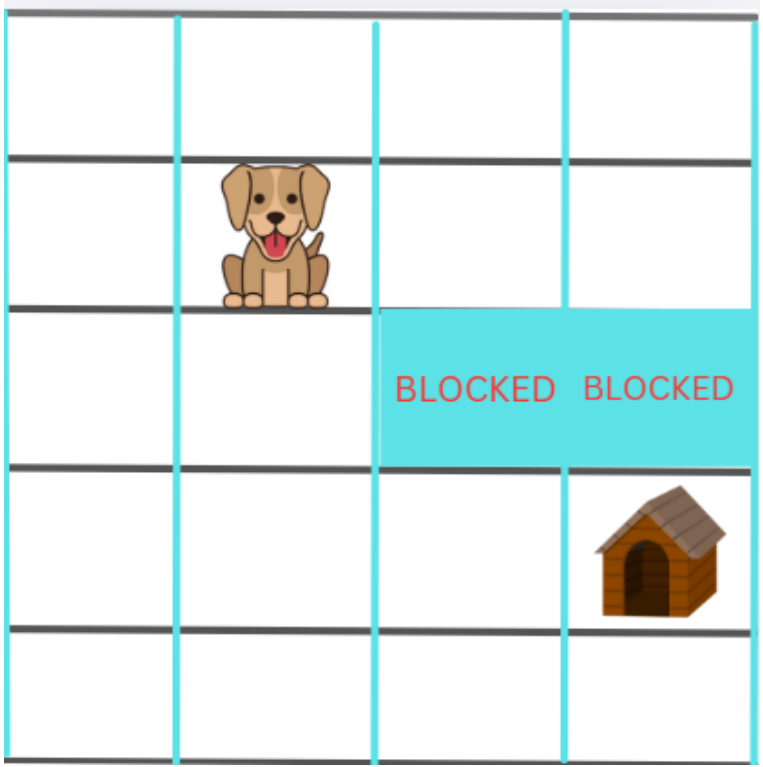
Never Overestimate the heuristic

Find Home for Kuki using A*?

G(n) = No of Step Taken?

H(n) =





Which method is Admissible?

Adversarial Search

• **Kasparov vs.DeepBlue(1997):** IBM's Deep Blue defeated world chess champion **Garry Kasparov** with 3 wins, 1 loss, and 2 draws, marking the first time a computer beat a champion in chess.

• **AlphaGo(GoogleDeepMind):** AlphaGo achieved superhuman performance in Go using deep learning and Monte Carlo Tree Search, demonstrating a major advance in AI beyond brute-force search.

Adversarial Search

- Used in **two-player competitive games**
- Goal: **choose the optimal move or strategy**
- Each player tries to **maximize their chance of winning**
- Assumes the **opponent plays optimally**
- Evaluates moves by considering the **opponent's best possible response**
- Selects the move that gives the **best outcome in the worst case**
- Commonly implemented using **Minimax (and Alpha–Beta pruning)**

Elements of a Game

- **S0:** The initial state, configuration of a chess board.
- **Successor Function:**
    - **PLAYER:** Two Players A & B
    - **ACTIONS:** Legal moves in a state.
    - **RESULT:** defines the result of a move.
- **TERMINAL-TEST:** End of Game?

Terminal test, which is true when the game is over and false otherwise. States where the game has ended are called terminal states.

- **UTILITY(s,p):** Utility function (an objective function) defines the final numeric value for a game that ends in terminal. In chess, the outcome is a win(+1), loss(-1), or draw(0).

ZERO-SUM Game

A zero-sum game is one where **one player's gain is exactly equal to the other player's loss**

- The **total payoff remains constant** throughout the game
- Any **increase in one player's payoff** results in a **decrease in the opponent's payoff**
- Players' objectives are **directly conflicting**
- Common examples: **Chess, Checkers, Nim**

Perfect Information (Zero-Sum) Game

- AI Games
  - **Perfect Information**
    - Deterministic
    - Fully Observable
  - **Zero Sum Game**
    - Utility Functions of each player at the end of the game are EQUAL and OPPOSITE
      e.g, WINNER (1) , LOSER (-1) or multi-valued utility values
  - Giving rise to adversary … **agents need to maximize their utility values**

Games vs. search problems

- **"Unpredictable" opponent**
  - specifying a move for every possible opponent reply
- **Time limits**
  - unlikely to find goal, must approximate

- **Nim** there are 48 possible positions, for **tic-tac-toe** around 5400 and for chess, well for **chess** there's a horribly big number of board positions possible.

Game tree (2-player, deterministic, turns)

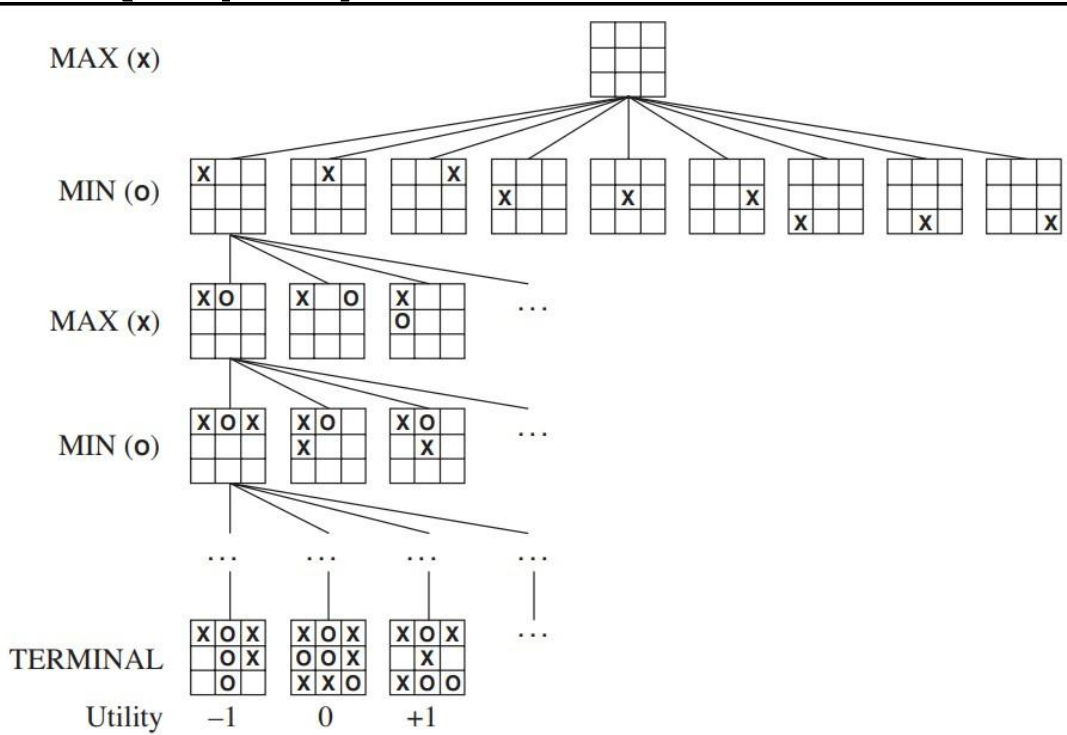


Figure 5.1 A (partial) game tree for the game of tic-tac-toe. The top node is the initial state, and MAX moves first, placing an X in an empty square. We show part of the tree, giving alternating moves by MIN (O) and MAX (X), until we eventually reach terminal states, which can be assigned utilities according to the rules of the game.

Optimal Decision in Games

- Search problem formulation for MINMAX
  - Initial State / Successor Function / Terminal State / Utility Fn
- Explanation
  - MAX starts first
  - Play alternates b/w MAX (places X) C MIN (places O)
  - Terminal State (WIN/LOSS/DRAW)
  - Number each leaf node w.r.t MAX
    - High values are GOOD for MAX, BAD for MIN
    - It is MAX's job to use search-tree to determine "best move"
- Normal Search solution vs. Game's Solution
  - Normal sequence of moves leading to goal vs. MIN has a say
  - Requires contingent OPTIMAL strategy (in response to MIN's

Minimax

- Perfect play for deterministic games
- Idea: choose move to position with **highest minimax value**=best achievable payoff against best play
- Back-tracking Algorithm
- Use DFS for searching
- Why not BFS?

Minimax

- Minimax Value=
  - **Utility (n)** if n is a terminal state
  - **Max(n)** set of successors if n is a MAX node
  - **Min(n)** set of successors if n is a MIN node

Minimax Search

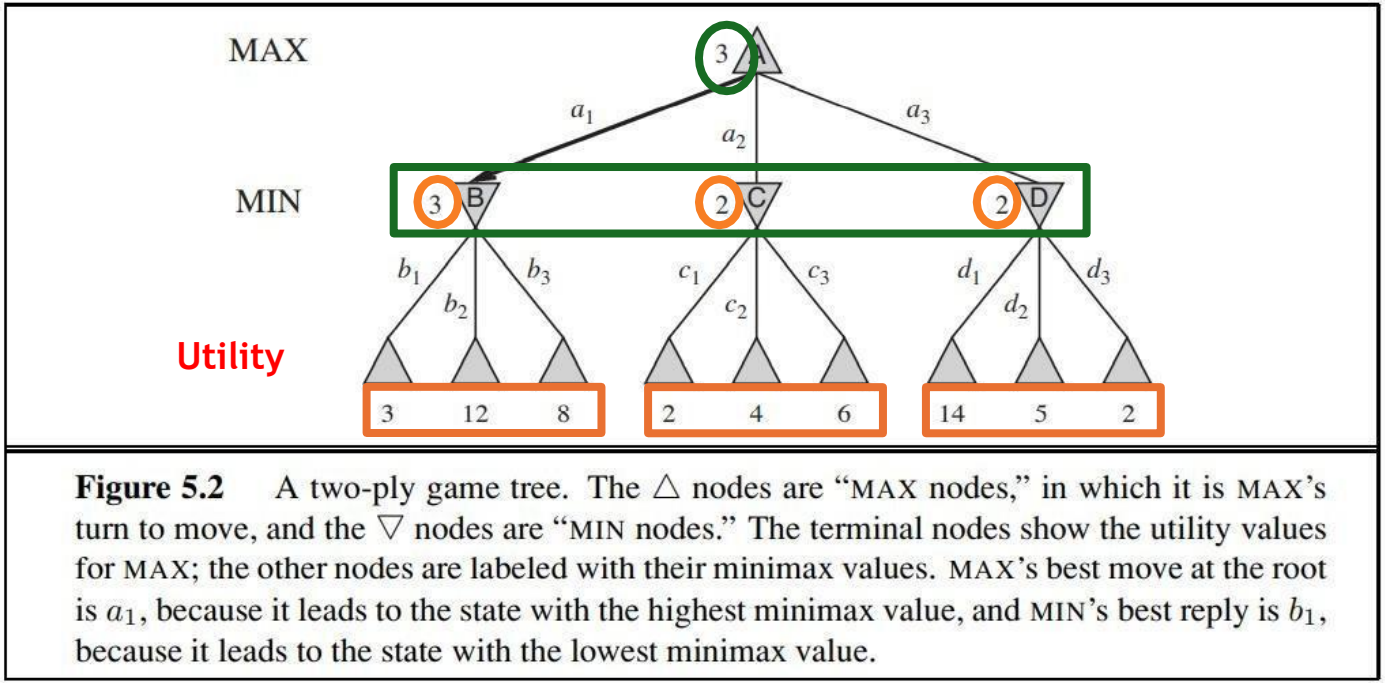


Figure 5.2 A two-ply game tree. The △ nodes are "MAX nodes," in which it is MAX's turn to move, and the ▽ nodes are "MIN nodes." The terminal nodes show the utility values for MAX; the other nodes are labeled with their minimax values. MAX's best move at the root is a_1, because it leads to the state with the highest minimax value, and MIN's best reply is b_1, because it leads to the state with the lowest minimax value.

Minimax Algorithm

```
function MINIMAX-DECISION(state) returns an action
  return arg max_{a ∈ ACTIONS(s)} MIN-VALUE(RESULT(state, a))

function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s, a)))
  return v

function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← ∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s, a)))
  return v
```

Figure 5.3 An algorithm for calculating minimax decisions. It returns the action corresponding to the best possible move, that is, the move that leads to the outcome with the best utility, under the assumption that the opponent plays to minimize utility. The functions MAX-VALUE and MIN-VALUE go through the whole game tree, all the way to the leaves, to determine the backed-up value of a state. The notation $\text{argmax}_{a \in S} f(a)$ computes the element a of set S that has the maximum value of $f(a)$.

MinMax

- **Complete**? Yes (if tree is finite)
- **Optimal**? Yes (against an optimal opponent)
- **Time complexity**? $O(b^d)$
- **Space complexity**? O(bd) (depth-first exploration)
- For chess, b ≈ 35, m ≈100 for "reasonable“ games
  - exact solution completely infeasible

MiniMax (Multiplayer)

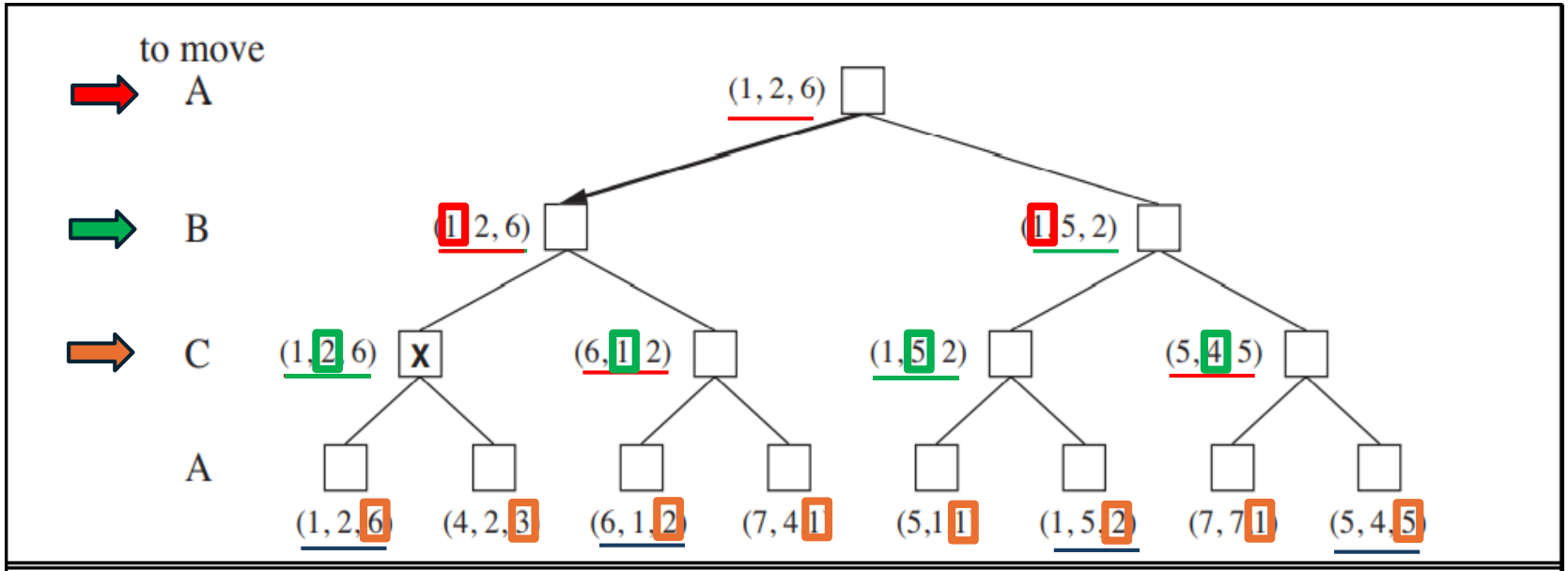


Figure 5.4 The first three plies of a game tree with three players (A, B, C). Each node is labeled with values from the viewpoint of each player. The best move is marked at the root.

MiniMax (Multiplayer)

- Multiplayer games usually involve alliances, whether formal or informal, among the players. Alliances are made and broken as the game proceeds
- Suppose A and B are in weak positions and C is in a stronger position. Then it is often optimal for both A and B to attack C rather than each other, lest C destroy each of them individually

MiniMax (Multiplayer)

- **Game Tree**: A tree consisting of nodes that represent the possible moves that each player can make in a two-player game.

- **Ply**: A single move by either the player or their opponent.

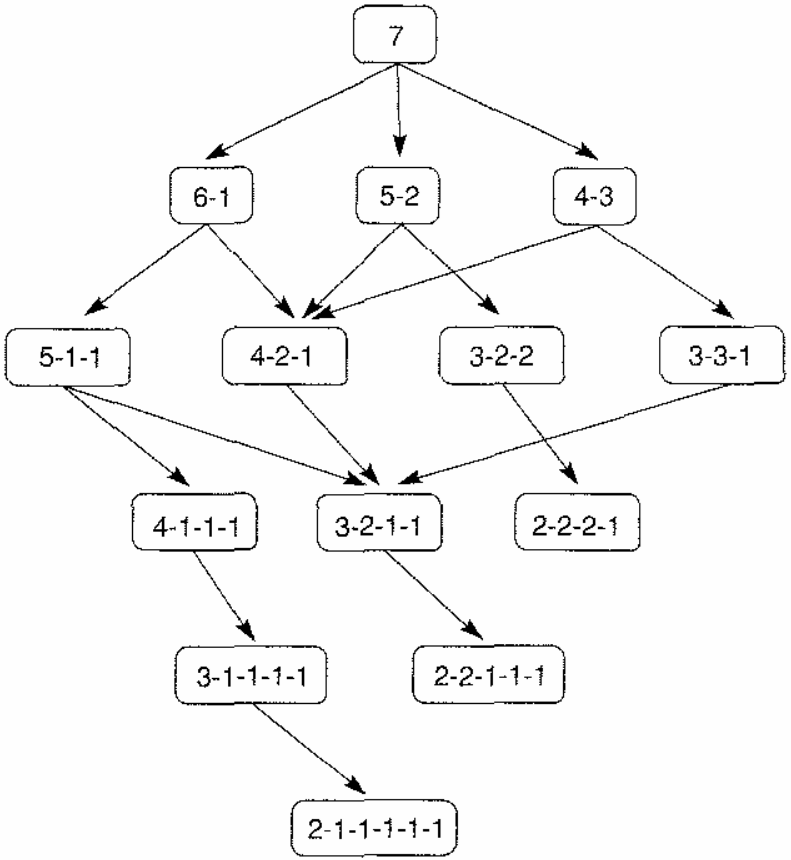
- **Move**: A set of plies, whereby each player makes a move.

Nim-7

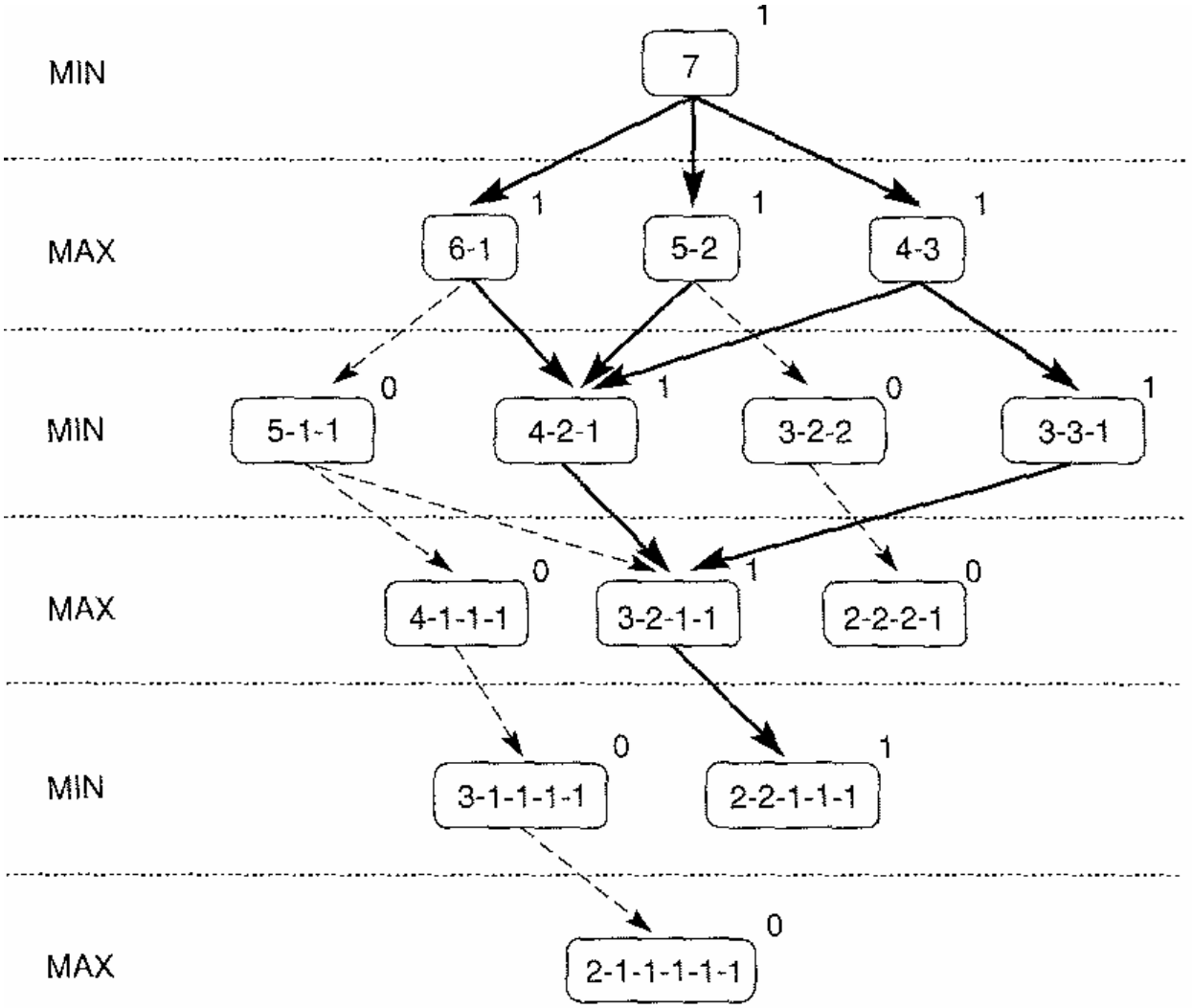
- The game starts with 7 coins in a single pile.
- There are two players, taking turns alternately.
- On each turn, a player must divide one pile into two unequal piles.
- The division must use all coins (no discarding).
- Example valid splits of 7: 1–6, 2–5, 3–4
- Equal splits are not allowed (e.g., 3–3 is invalid).
- Once piles are created, any pile may be chosen on later turns for further unequal division.
- A player who cannot make a move loses (cannot divide further).

Complete game tree for Nim-7

- 7 coins are placed on a table between the two opponents
- A move consists of dividing a pile of coins into two nonempty piles of different sizes
- For example, 6 coins can be divided into piles of 5 and 1 or 4
  and 2, but not 3 and 3
- The first player who can no longer make a move loses the game

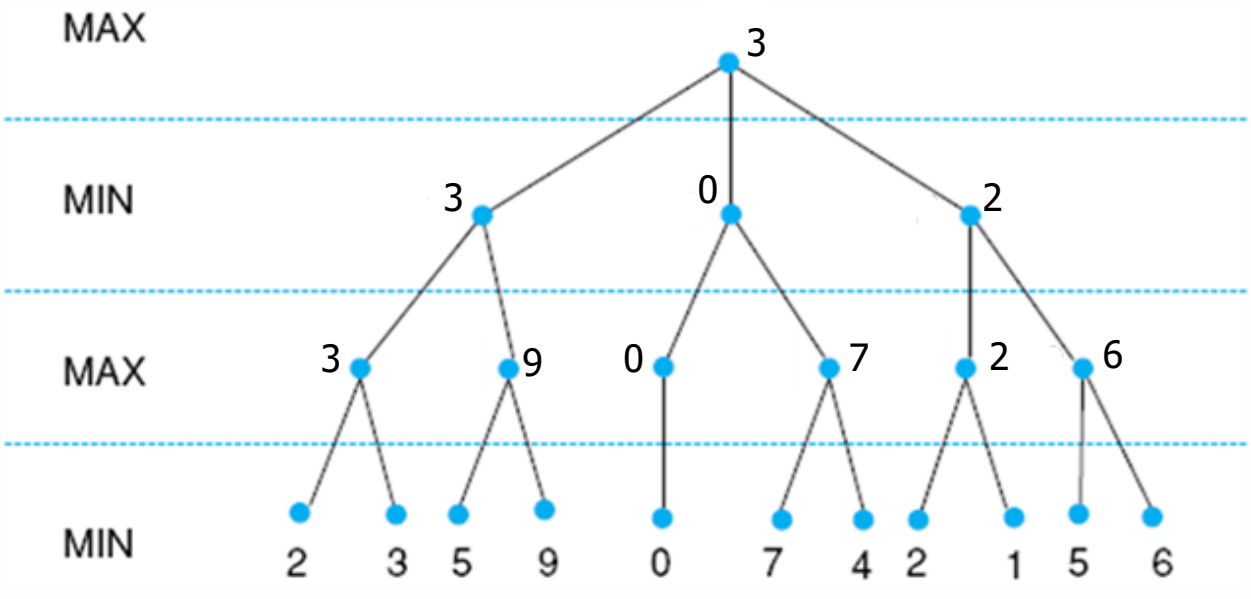


MinMax on Nim



Minimax to a Hypothetical State space

- Leaf values show heuristic values;
- Internal states show backed-up values



Tic-Tac-Toe

- Tic-Tac-Toe is a 3×3 board → 9 cells.
- Each cell can be:
  - Empty (0)
  - X (1)
  - O (2)
- Maximum total raw board configurations = 3^9 =19,683
- This counts all arrangements, including illegal ones (e.g., 5 X's and
  1 O).

Tic-Tac-Toe

- After removing illegal and duplicate positions, there are:
- Total legal board states: ≈ **5,478**
- Total possible games (paths through the game tree): ≈ **255,168**

Tic-Tac-Toe – Ply & Heuristic Evaluation

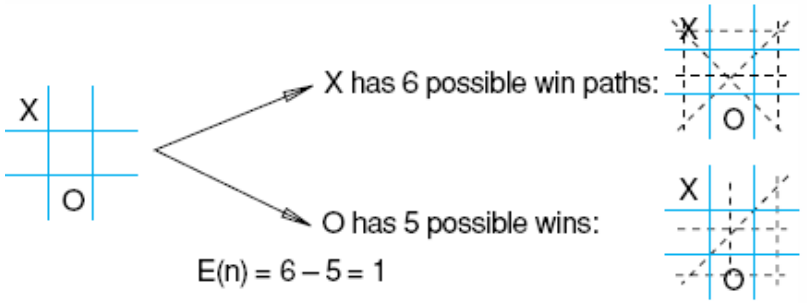
- **Game tree:** all possible moves from the current board
- **Full expansion** is impractical → use **fixed ply depth look-ahead**
- **Example**: 2 plies = your move + opponent's move
- **Generate future board states** up to the chosen ply depth
- **Evaluate each board** using the heuristic:

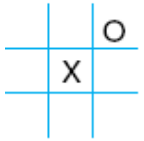
$$E(n)=M(n)-O(n)$$

  - M(n) = total number of player 1's potential winning lines
  - O(n) = total number of opponent's potential winning lines
  - E(n) = evaluation score for board state n
- Propagate scores back to the current move
- Choose move with the highest evaluation **E(n)**

Heuristic measuring conflict applied to states of tic-tac- toe

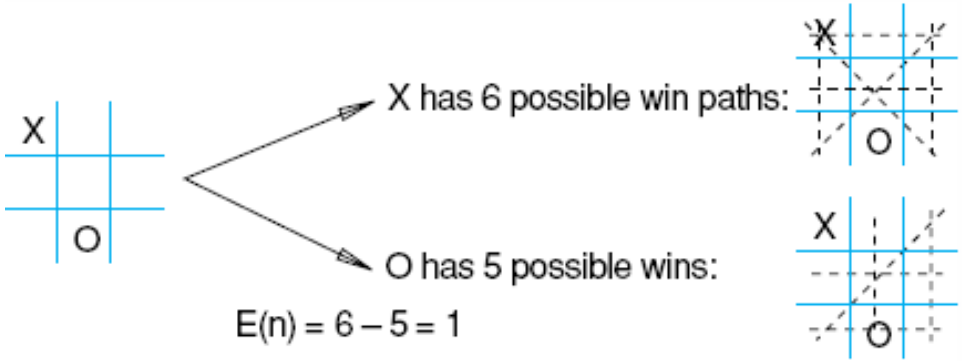
- Maximize E(n)
- E(n) = 0 when my opponent and I have equal number of possibilities.





Heuristic is E(n) = M(n) – O(n)
where M(n) is the total of My possible winning lines
O(n) is total of Opponent's possible winning lines
E(n) is the total Evaluation for state n

Heuristic measuring conflict applied to states of tic-tac- toe



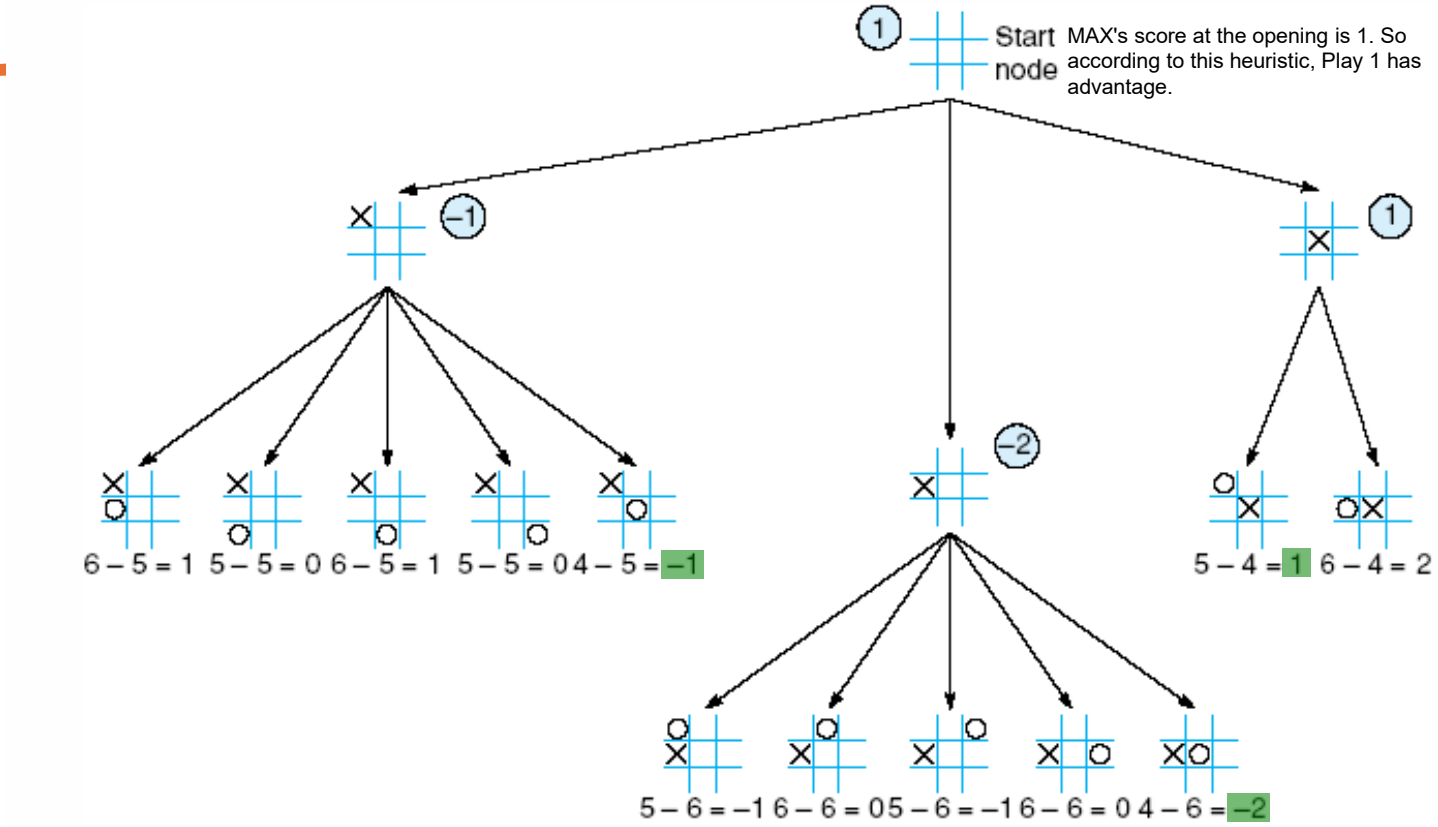
- Maximize E(n)
- E(n) = 0 when my opponent and I have equal number of possibilities.

X		
		0

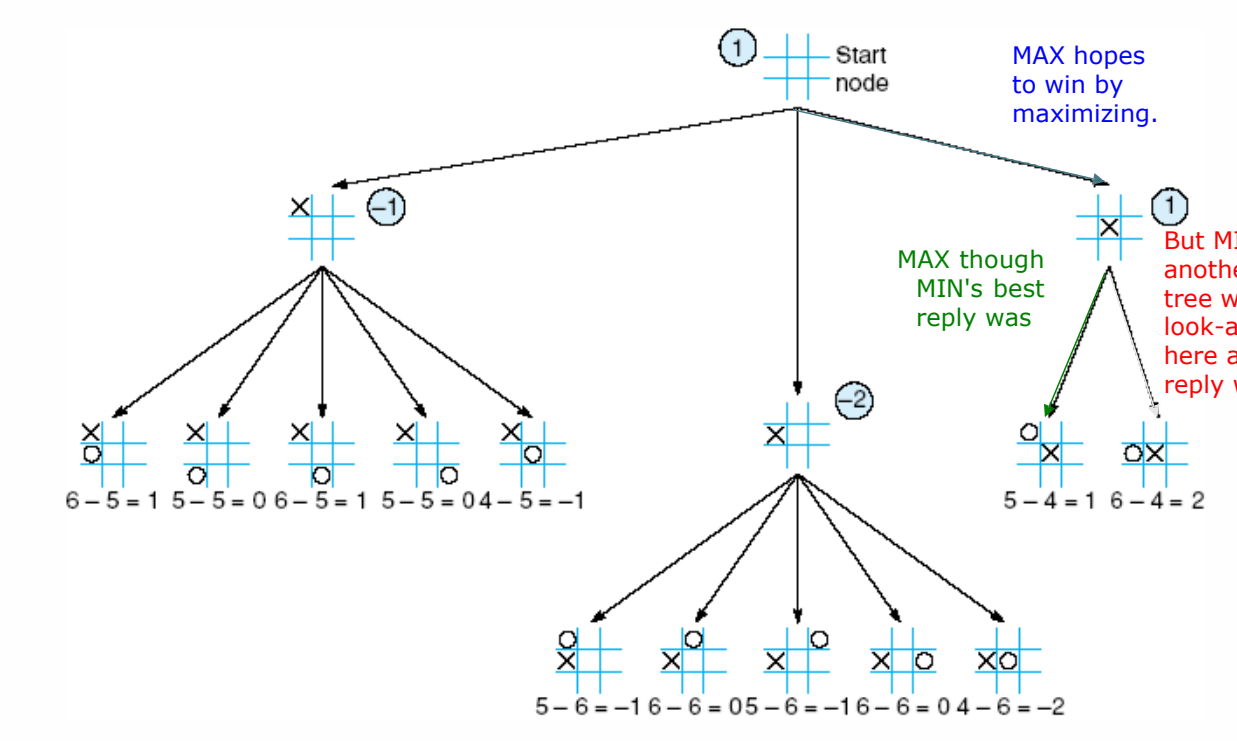
Compute E(n) =?

E(n) = 6-5 = 1

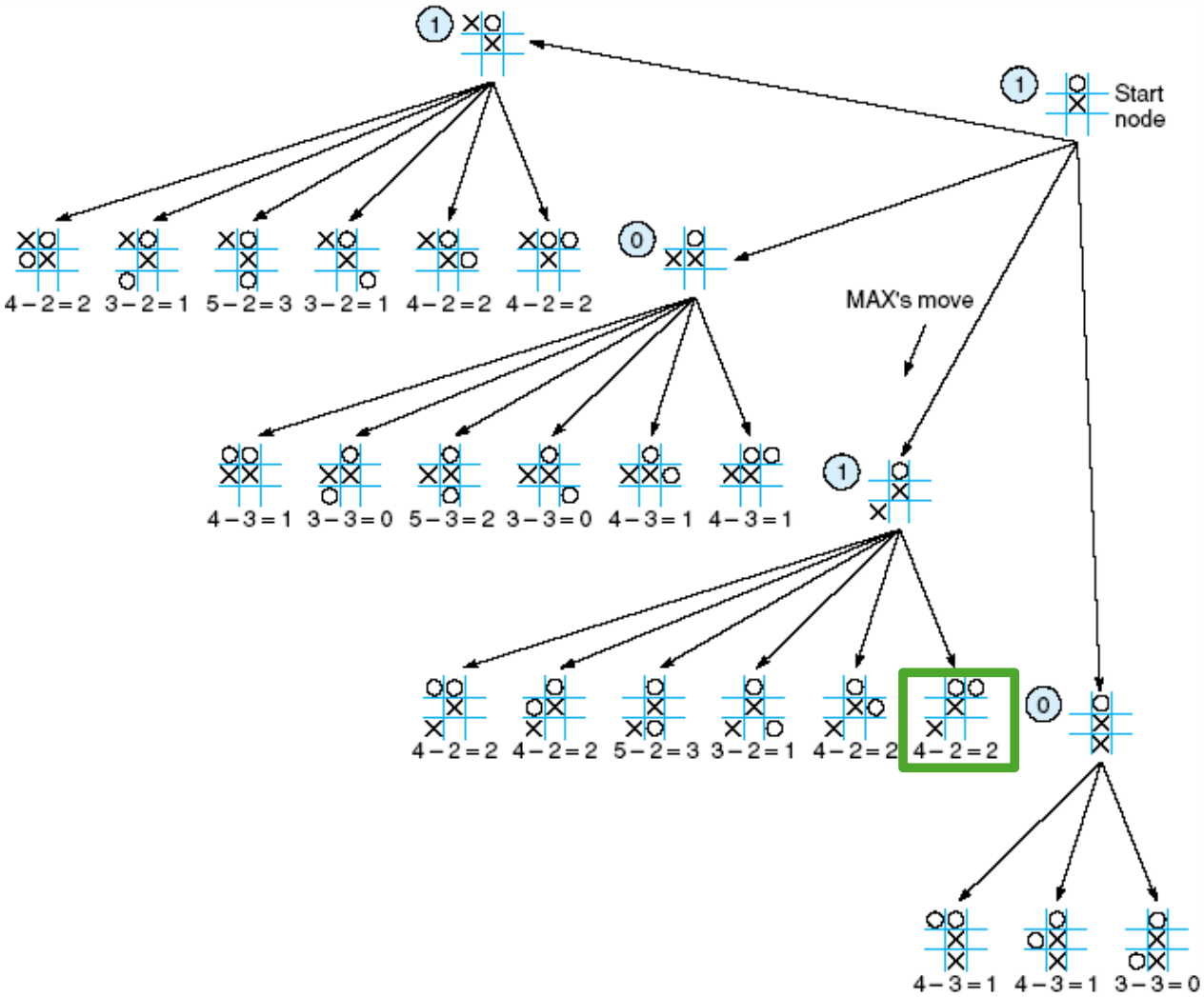
Tic-tac-toe, MAX vs MIN, 2- ply look-ahead



MAX make his first move

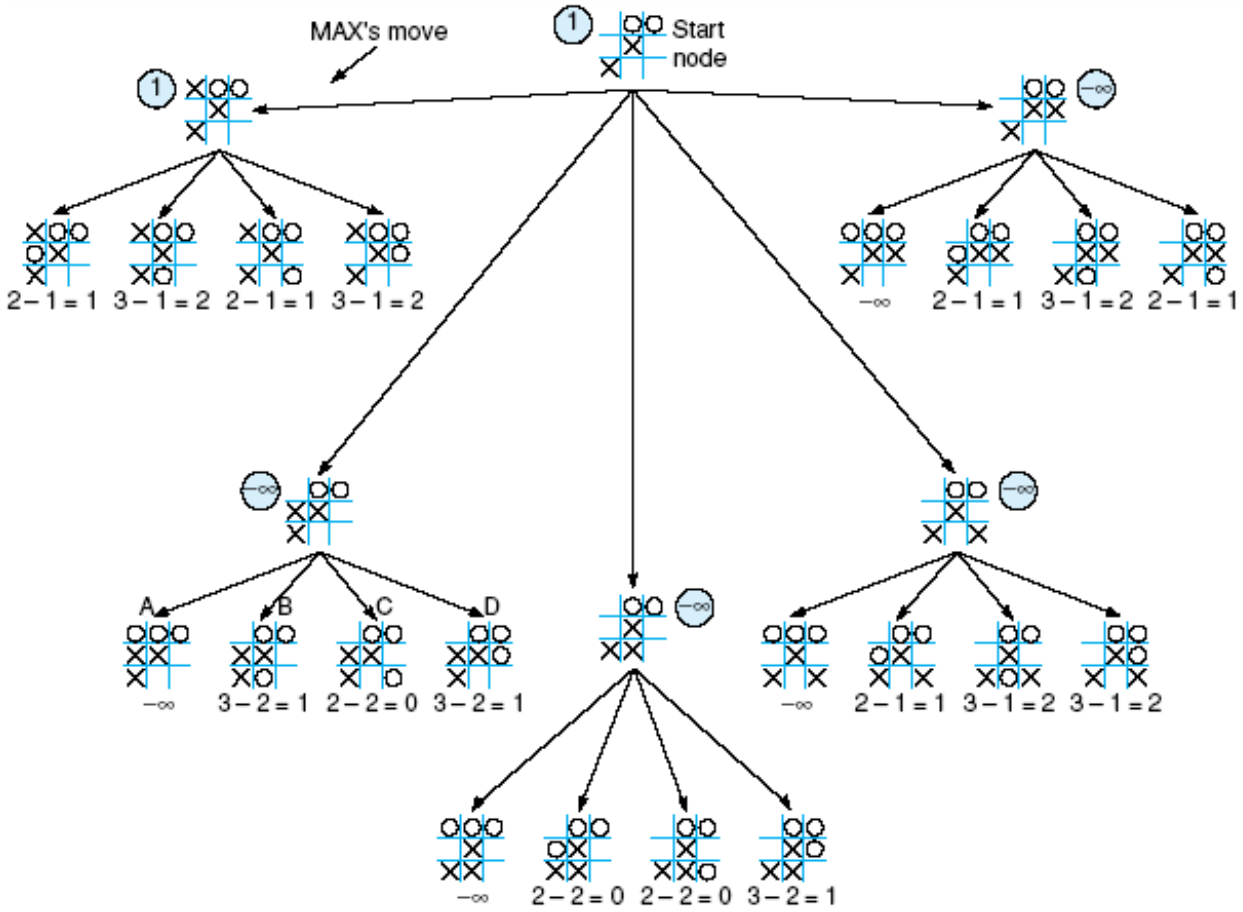


MAX's 2nd move: look ahead analysis



MAX's 3rd move: look ahead analysis

−∞ means MIN wins.
+∞ means MAX wins.



Question?

- How to improve search efficiency?
- It is possible to cut off some unnecessary subtrees?

Alpha Beta Pruning

It is possible to compute the correct minimax decision w**ithout looking at every node in the game tree.**

Alpha-beta pruning allows **to eliminate large parts of the tree from consideration, without influencing the final decision.**

Alpha-beta pruning gets its name from two parameters.

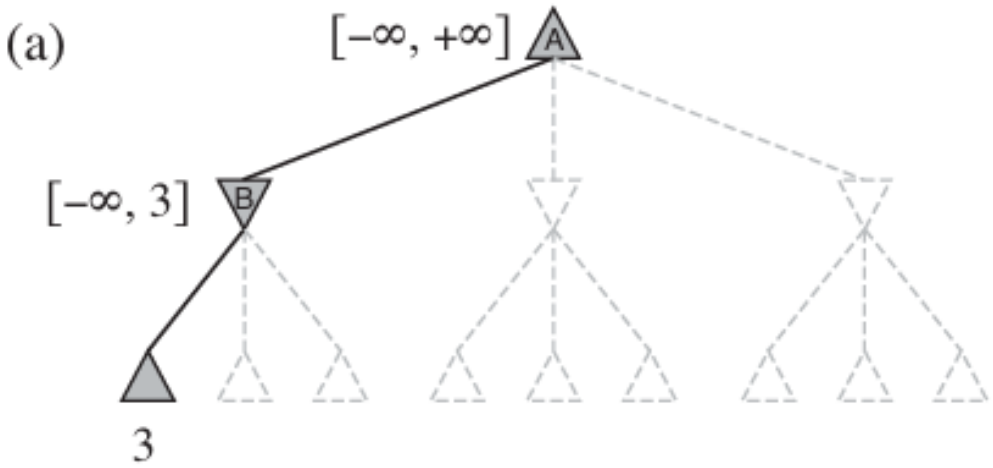
- They describe bounds on the values that appear anywhere along the path under consideration:
- **α =** the value of the best (i.e., **highest value)** choice found so far along the path for MAX
- **β =** the value of the best (i.e., **lowest value)** choice found so far along the path for MIN

Alpha Beta Pruning

The leaves below B have the values 3, 12 and 8.

The value of B is exactly 3.

It can be inferred that the value at the root is at least 3, because MAX has a choice worth 3.

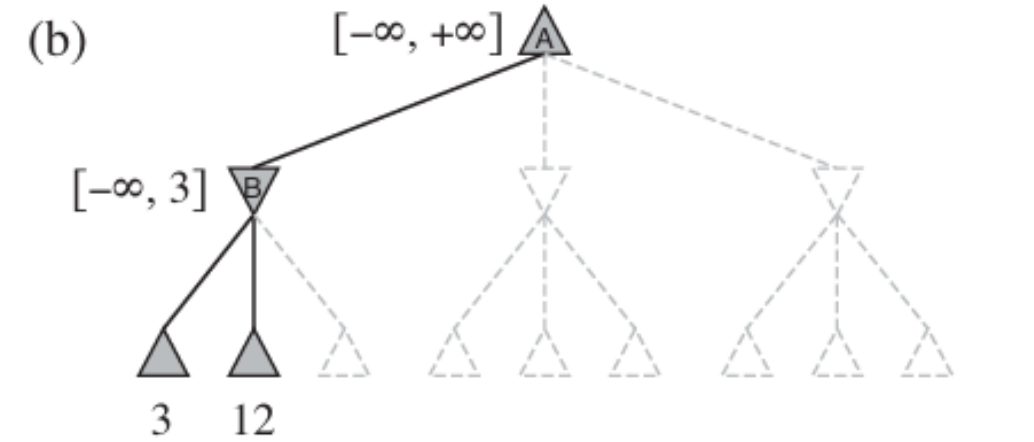


Alpha Beta Pruning

The leaves below B have the values 3, 12 and 8.

The value of B is exactly 3.

It can be inferred that the value at the root is at least 3, because MAX has a choice worth 3.

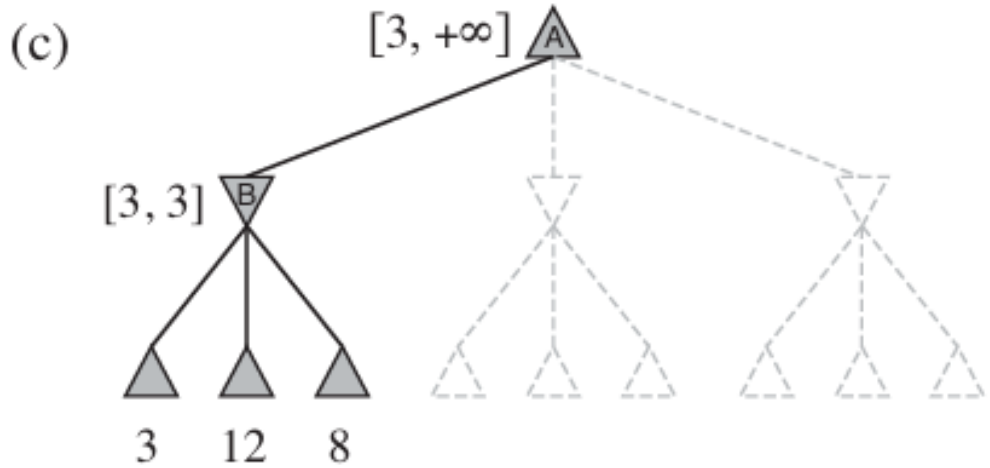


Alpha Beta Pruning

The leaves below B have the values 3, 12 and 8.

The value of B is exactly 3.

It can be inferred that the value at the root is at least 3, because MAX has a choice worth 3.

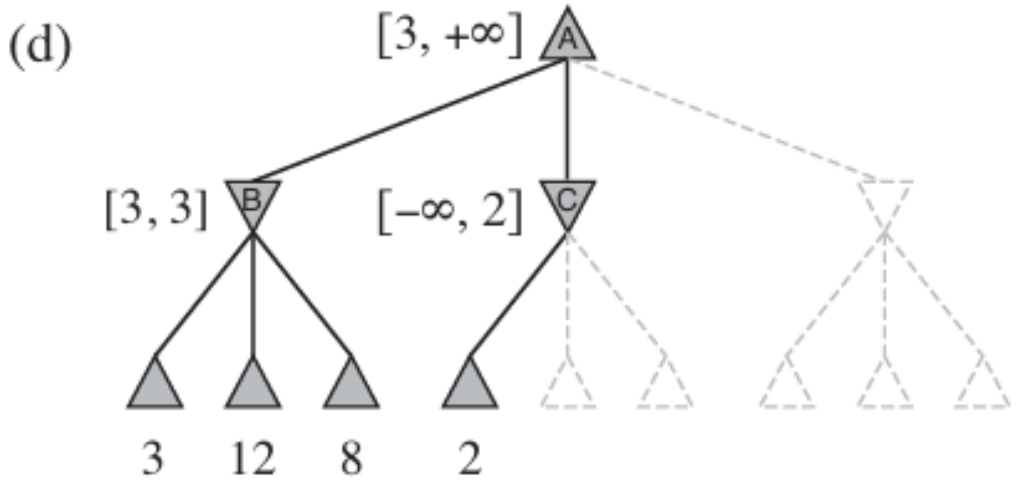


Alpha Beta Pruning

The leaves below B have the values 3, 12 and 8.

The value of B is exactly 3.

It can be inferred that the value at the root is at least 3, because MAX has a choice worth 3.

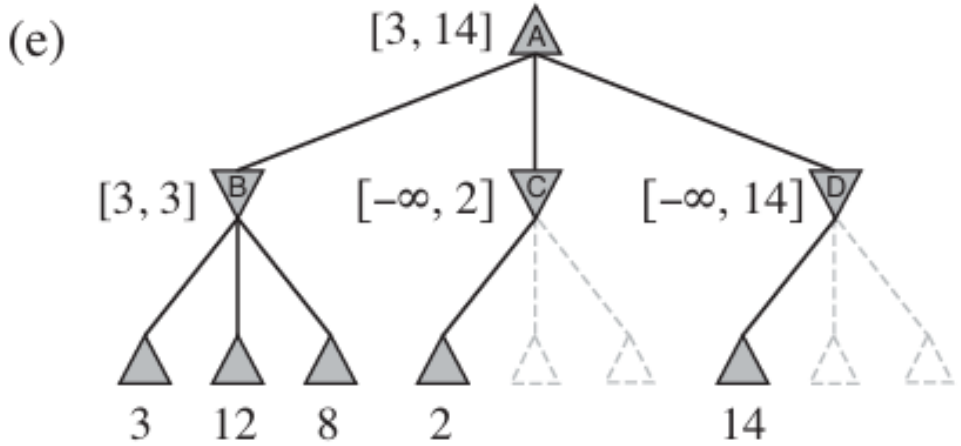


Alpha Beta Pruning

The leaves below B have the values 3, 12 and 8.

The value of B is exactly 3.

It can be inferred that the value at the root is at least 3, because MAX has a choice worth 3.

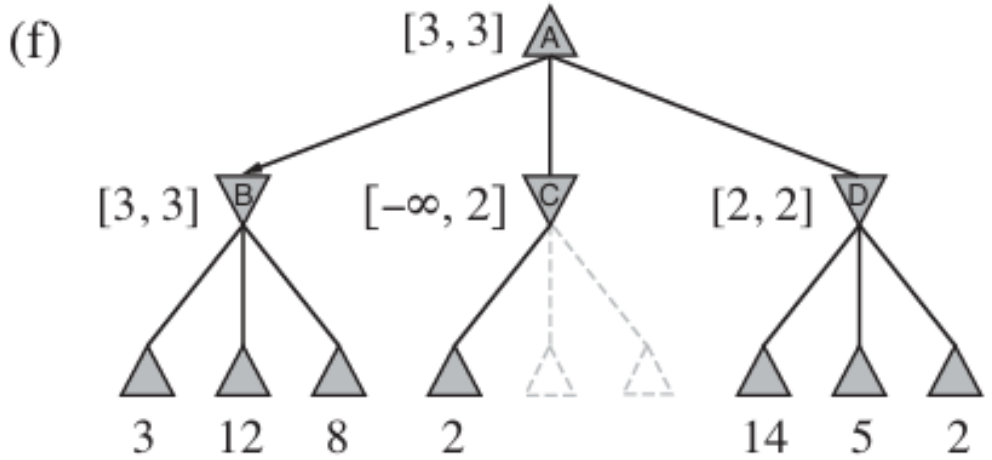


Alpha Beta Pruning

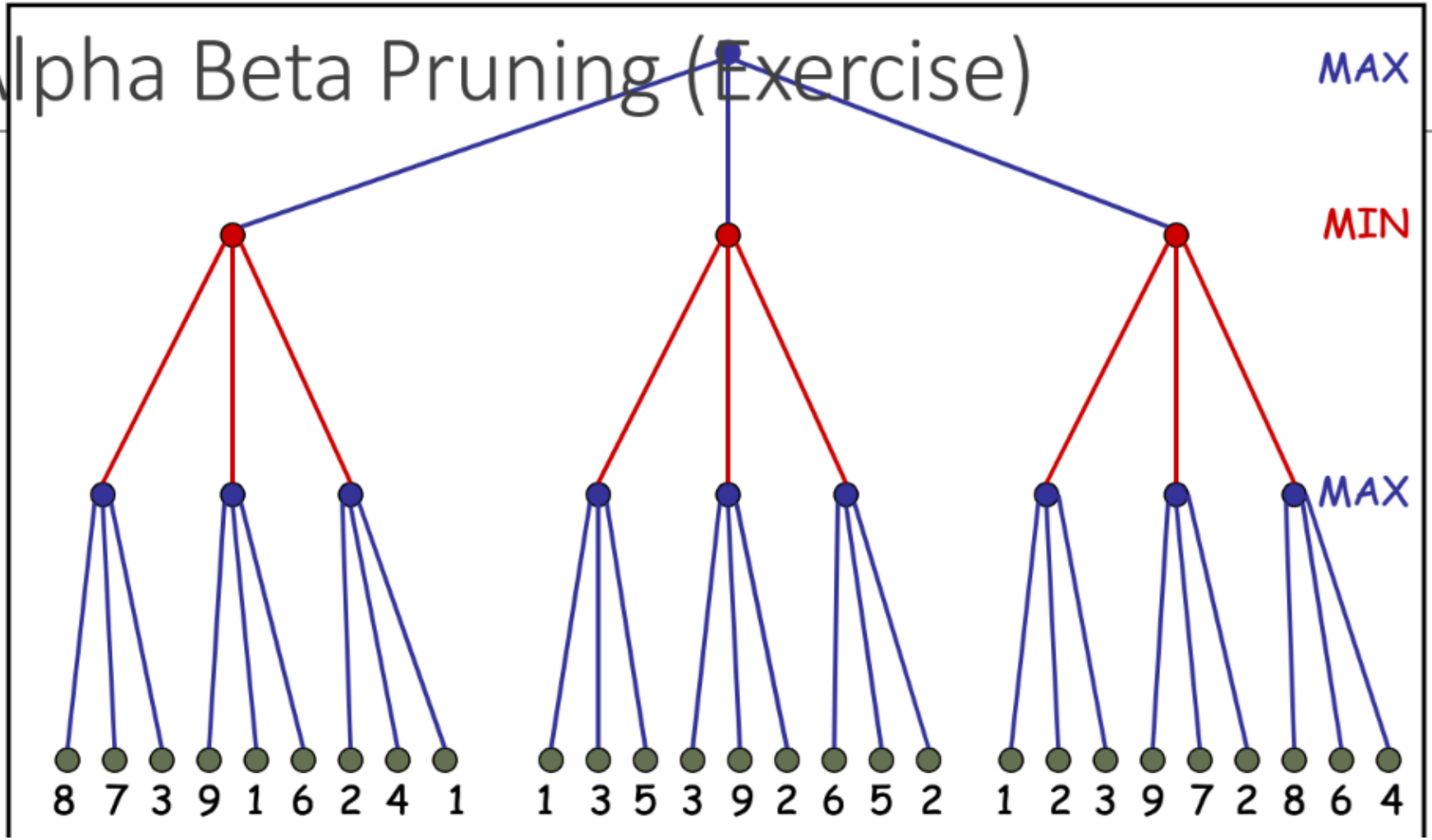
The leaves below B have the values 3, 12 and 8.

The value of B is exactly 3.

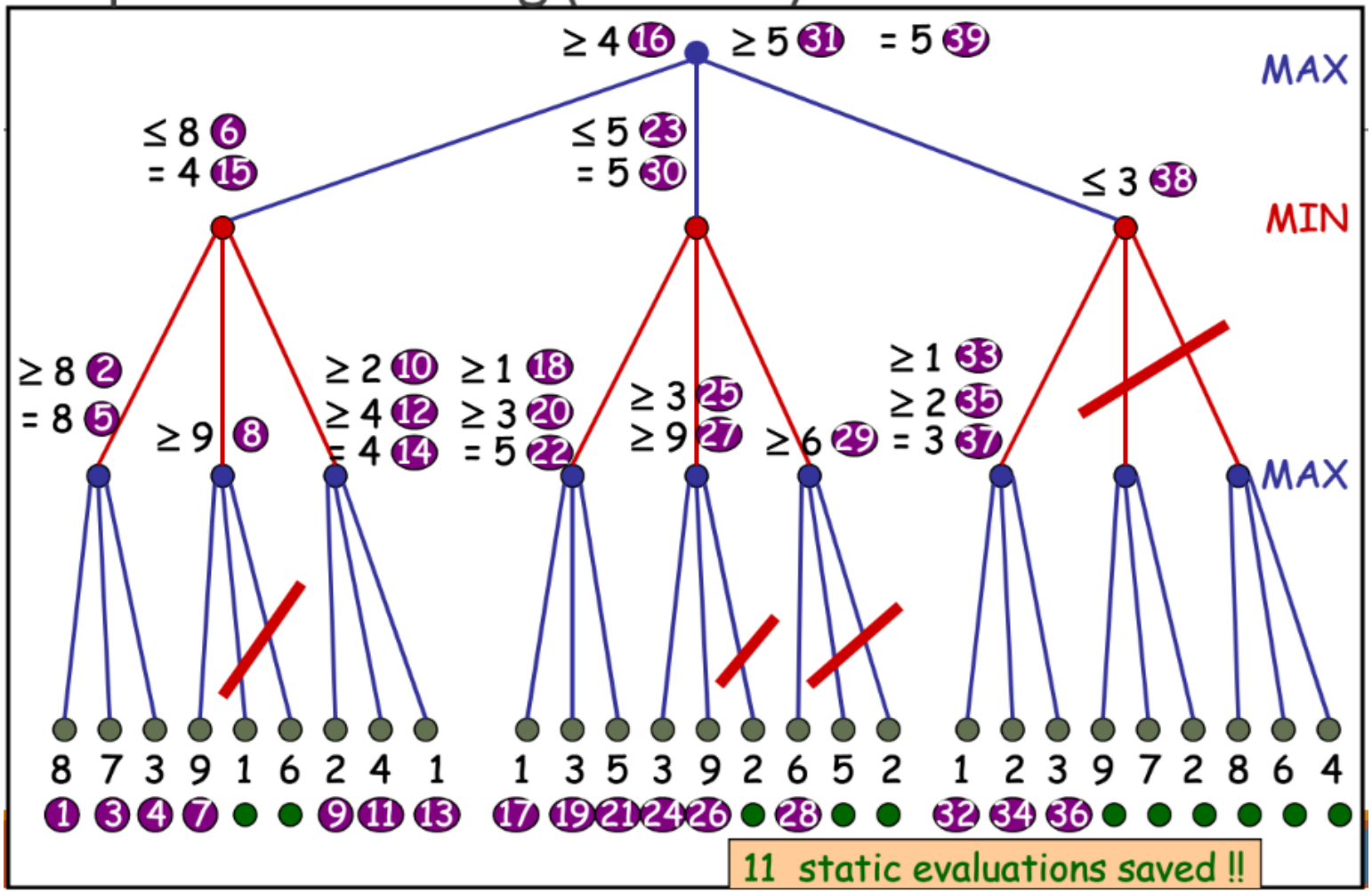
It can be inferred that the value at the root is at least 3, because MAX has a choice worth 3.



Alpha Beta Pruning (Exercise)



Alpha Beta Pruning (Exercise)



Question?

Perform min-max search with $\alpha-\beta$ pruning in the tree below. Enter values at the nodes as soon as you know something. Show the branches which are cut, and the nodes in the bottom layer which are not evaluated. The top node is a max node.